

A Zero-Shot Framework for Cross-Project Vulnerability Detection in Source Code

Radowanul Haque¹ [ORCID](#), Aftab Ali¹ [ORCID](#), Sally McClean¹ [ORCID](#), Naveed Khan¹ [ORCID](#)

¹*School of Computing, Ulster University, BT15 1ED, Belfast, U.K.*

Corresponding author: haque-r1@ulster.ac.uk

Contributing authors: a.ali@ulster.ac.uk; si.mcclean@ulster.ac.uk; n.khan@ulster.ac.uk

Acknowledgement

This research is supported by the ARC (Advanced Research Engineering Centre) project. PwC* is in receipt of Grant for R&D support from Invest NI for ARC. This project is part-financed by the European Regional Development Fund under the Investment for Growth and Jobs Programme 2014-2020.

*PricewaterhouseCoopers LLP a limited liability partnership incorporated in England with its registered office at 1 Embankment Place, London WC2N 6RH

Abstract

The growing prevalence of software vulnerabilities necessitates robust detection mechanisms, particularly in cross-project settings, where domain discrepancies pose significant challenges. Existing vulnerability detection models often struggle to generalise across projects due to differences in coding styles, feature distributions, and the absence of labelled target data. To address these challenges, this paper introduces ZSVulD, a zero-shot, cross-project vulnerability detection framework that enhances generalisation across unseen software projects without requiring labelled target data. ZSVulD leverages domain-agnostic CodeBERT embeddings to capture both syntactic and semantic features of source code, facilitating effective knowledge transfer. The framework employs an iterative pseudo-labelling mechanism, where neural networks and XGBoost collaboratively refine predictions, progressively improving target domain classification. Additionally, feature alignment techniques are integrated to reduce discrepancies between source and target datasets, ensuring consistency across diverse projects. Comprehensive experiments on two widely used vulnerability detection datasets, Devign and REVEAL, demonstrate the effectiveness of ZSVulD. The framework outperforms state-of-the-art methods, achieving higher recall, F1 scores, and F2 scores. This minimises false negatives, which is crucial in security-critical applications. These results highlight ZSVulD's potential for practical deployment in automated vulnerability detection pipelines, enabling more reliable security assessments across diverse software ecosystems.

Keywords: LLMs, Vulnerability Detection, Code Classification, Zero-Shot Learning

1 Introduction

The increasing reliance on software in critical systems has brought heightened attention to the discovery and mitigation of software vulnerabilities. Vulnerability detection, the process of identifying weaknesses or flaws in software that could be exploited to compromise security, integrity, or functionality, has become a cornerstone of research in software security. These vulnerabilities pose significant risks, ranging from data breaches to system failures, making their detection critical to protecting sensitive data and ensuring system reliability (Chakraborty et al. 2022). According to a 2024 study by IBM, the global average cost of a data breach reached \$4.88 million, with breaches in critical infrastructure sectors incurring even higher costs (IBM 2024).

Recent surveys (Kaur and Nayyar 2020a; Zeng et al. 2020; Chakraborty et al. 2022; Harzevili et al. 2023) on software vulnerability detection provides a comprehensive overview of traditional and modern techniques, from static and dynamic analysis to machine learning-based approaches. Static analysis tools like Flawfinder (David A. Wheeler 2017) rely on rule-based heuristics, while dynamic analysis tools such as fuzzing identify vulnerabilities by monitoring software execution (Kaur and Nayyar 2020a). However, these methods often face challenges related to scalability and the generation of false positives, necessitating the exploration of advanced, learning-based techniques. The survey by Lin et al. (Lin et al. 2020) delves into deep neural network applications for vulnerability detection, highlighting their potential for learning complex patterns in source code. Similarly, a systematic review by Harzevili et al. (Harzevili et al. 2023) discusses machine learning advancements in automated vulnerability detection, emphasising the importance of dataset quality, feature engineering, and explainability. Marjanov et al. (Marjanov et al. 2022) analyse what works well in machine learning for source code vulnerability detection and identify gaps in current approaches, such as addressing complex code semantics and dataset biases.

While within-project vulnerability detection has demonstrated success by training and testing models on data from the same project, it falls short when applied to new, unseen projects. This limitation arises from significant domain shifts, such as differences in coding styles, library usage, and structural complexities, which hinder the generalisability of within-project models to cross-project scenarios (Li et al. 2023).

Zero-shot and few-shot learning in Cross-Project vulnerability Detection (CPVD) (Nguyen et al. 2020, 2024; Li et al. 2023) have gained significant attention as promising strategies to address these challenges. CPVD aims to train models on one or more source projects and apply them to detect vulnerabilities in the target without requiring labelled target data, making it particularly well-suited for zero-shot learning (Li et al. 2023). However, this task is inherently complex due to domain discrepancies between source and target datasets arising from differences in coding practices, libraries, and project-specific conventions.

Recent studies (Lin et al. 2018; Nguyen et al. 2020, 2024; Liu et al. 2022; Li et al. 2023; Zhang et al. 2023; Cai et al. 2024) have explored various domain adaptation techniques to bridge these gaps, leveraging transfer learning, adversarial feature alignment, and graph-based neural networks to capture and transfer vulnerability-relevant patterns. Despite these advancements, existing methods often face three major limitations: (1) difficulty in aligning feature distributions between source and target projects, (2) noisy pseudo-labels that hinder adaptation performance, and (3) inefficient integration of syntactic and semantic representations, reducing their ability to generalise effectively in zero-shot settings.

This paper introduces ZSVulD, a novel framework for zero-shot, cross-project vulnerability detection to address these challenges. Unlike existing methods, ZSVulD leverages CodeBERT embeddings, which are domain-agnostic and capture the structure and meaning of source code, making them well-suited for generalisation across diverse software projects. These embeddings are processed using an iterative pseudo-labelling approach, where neural networks and XGBoost collaboratively refine predictions for the target dataset. The framework also utilises feature alignment techniques to mitigate domain discrepancies, including normalisation and iterative fine-tuning. By aligning feature distributions, ZSVulD ensures that semantically similar code functions from different projects are mapped closer in the feature space, thereby enhancing generalisation and robustness, even in the absence of labelled target data.

To evaluate the effectiveness of the proposed framework, we define the following research questions:

- *RQ1*: How does fine-tuned CodeBERT perform in cross-project vulnerability detection?
- *RQ2*: How effectively does the proposed framework generalise to unseen target projects in a zero-shot setting, where no labelled target data is available?
- *RQ3*: How well do CodeBERT embeddings capture vulnerability-relevant features, and how effectively does the framework align source and target feature distributions?
- *RQ4*: How does the proposed framework compare to state-of-the-art cross-project vulnerability detection methods in terms of recall, precision, F1-score, and F2-score?

The main contributions of this paper are as follows:

1. **Evaluation of Fine-Tuned CodeBERT in Cross-Project Settings:** We analyse how fine-tuning CodeBERT on individual datasets impacts its performance in cross-project scenarios, revealing its generalisation limitations.

2. **A Novel Zero-Shot Framework for Cross-Project Vulnerability Detection:** We propose ZSVulD, which integrates CodeBERT embeddings, pseudo-labelling, and ensemble learning to improve prediction performance without requiring labelled target data.
3. **Feature Alignment for Domain Bridging:** We employ normalisation and iterative fine-tuning techniques to mitigate feature distribution differences, ensuring more effective generalisation across projects.
4. **Comprehensive Evaluation Against State-of-the-Art Methods:** We benchmark ZSVulD against existing cross-project vulnerability detection approaches, demonstrating its superior recall, F1-score, and F2-score, particularly in minimising false negatives, which is critical for security applications.

The remainder of this paper is structured as follows: Section 2 presents the literature review, Section 3 details the proposed framework, Section 4 describes the experimental settings, Section 5 presents the results and discussion, Section 6 describes threats to validity, and Section 7 concludes with future works.

2 Related Works

The application of machine learning (ML) and deep learning (DL) techniques has transformed vulnerability detection in source code. These approaches have shifted the paradigm from static, rule-based methods to data-driven frameworks capable of uncovering complex patterns indicative of vulnerabilities. Two prominent paradigms have emerged in this context: within-project detection, which focuses on identifying vulnerabilities within a single project, and cross-project detection, which seeks to generalise detection capabilities across diverse projects. This section reviews existing research, emphasising the contributions, methodologies, and challenges in leveraging ML and DL for these tasks.

2.1 Within-Project Vulnerability Detection

Vulnerability detection in source code is a critical aspect of software security, aiming to identify weaknesses that could be exploited by attackers. Recent advancements in machine learning (ML), deep learning (DL), natural language processing (NLP), and graph-based learning have significantly improved the precision and scalability of these techniques.

Traditionally, vulnerability detection relied heavily on rule-based static and dynamic analysis methods (Kaur and Nayyar 2020b). While effective in specific contexts, these methods are often limited by their inability to generalize across diverse codebases and their susceptibility to false positives. Recent research addresses these limitations by incorporating data-driven approaches, where machine learning models learn patterns from labelled datasets. For example, SySeVR, introduced by Li et al. (Li et al. 2022), proposes a framework leveraging deep learning to analyse code slices and capture vulnerability-related patterns. The framework integrates tokenization, slicing, and graph-based representations to achieve state-of-the-art performance in detecting vulnerabilities within projects.

The integration of deep learning models has revolutionized vulnerability detection by enabling models to automatically extract meaningful representations from raw source code. Transformer-based models, such as BERT (Devlin et al. 2019) and its variants tailored for code such as CodeBERT (Feng et al. 2020), have demonstrated exceptional performance. These models leverage pre-training on large-scale code datasets, capturing nuanced language semantics that traditional approaches often miss. For instance, methods like VULREM (Gürfidan 2024) and VulBERTa (Hanif and Maffeis 2022) fine-tune pre-trained models on

project-specific data, enabling high precision and recall for within-project vulnerability detection. Compared to traditional approaches, transformer models offer superior adaptability and scalability, though their reliance on large pre-training datasets and high computational resources may present challenges.

Graph-based learning methods have also made significant contributions to vulnerability detection, representing source code as Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), or Data Flow Graphs (DFGs). These graphs capture structural and contextual relationships within the code, providing rich input for neural networks. For instance, frameworks like DiverseVul (Chen et al. 2023) combine graph-based representations with language models, enhancing the model's ability to detect vulnerabilities while retaining the project's structural integrity (Ma et al. 2022). Compared to transformer models, graph-based approaches excel in encoding structural patterns, such as data and control flows, which are often critical for identifying vulnerabilities in low-level programming languages like C or C++. However, graph-based methods may struggle to capture semantic nuances, making them less effective for projects where vulnerabilities stem from subtle logical errors or contextual dependencies.

Emerging technologies, such as quantum natural language processing (QNLP), further exemplify innovation in this domain. Akter et al. (Akter et al. 2023) explore the application of QNLP for automated vulnerability detection, demonstrating the potential of quantum computing to enhance the efficiency and accuracy of analysing complex code structures. While still in its infancy, QNLP represents a promising direction for future research, particularly in the context of highly intricate or large-scale systems.

Static analysis remains a cornerstone of vulnerability detection, particularly when augmented with ML techniques. For example, the work by Marjanov et al. (Marjanov et al. 2022) examines the current state of machine learning applications in source code vulnerability detection, identifying effective strategies and areas that require further development. While static analysis excels in identifying known patterns and vulnerabilities with deterministic signatures, its integration with ML models enables broader detection capabilities, particularly for previously unseen vulnerability patterns.

While within-project vulnerability detection has shown promising results in identifying vulnerabilities within a single project, it faces significant limitations when applied to real-world scenarios (Nguyen et al. 2020). The primary drawback is its reliance on project-specific labelled data, which may not always be available or sufficient for training robust models (Li et al. 2023). Additionally, models trained on one project often fail to generalise effectively to other projects due to differences in coding styles, library dependencies, and structural patterns. This lack of cross-project generalisation limits the practical applicability of within-project methods, highlighting the need for more advanced approaches capable of detecting vulnerabilities across diverse software projects without requiring labelled data for each new target project.

2.2 Cross-Project Vulnerability Detection

Cross-Project Vulnerability Detection (CPVD) extends the capabilities of within-project approaches by addressing the challenges posed by domain discrepancies in software security. Unlike within-project detection, which relies on homogeneous datasets, CPVD focuses on generalising detection models to work across diverse software environments (Zhang et al. 2023). These environments often exhibit significant variability in coding practices, project-

specific patterns, and library usage, requiring models to overcome domain shifts to perform effectively on unseen codebases. CPVD is particularly valuable in real-world scenarios where labelled data for target projects is scarce or unavailable, making the development of domain-agnostic solutions a critical research focus (Zhang et al. 2023).

Representation learning has played a pivotal role in CPVD methodologies by enabling models to extract transferable knowledge from source projects and apply it to target domains. For instance, Dam et al. (Lin et al. 2018) proposed a transfer learning framework that encodes universal patterns from vulnerable and non-vulnerable code into a shared representation space. Their approach demonstrated the feasibility of generalising vulnerability detection to unseen projects with minimal reliance on labelled data, emphasising the importance of domain-agnostic representations for effective cross-project adaptation.

Graph-based Neural Networks (GNNs) have emerged as powerful tools for CPVD due to their ability to model both structural and semantic relationships in source code. Zhang et al. (Zhang et al. 2023) introduced a framework that integrates Graph Attention Networks (GATs) with domain adaptation techniques, achieving feature distribution alignment between source and target projects. Their approach effectively captures the structural intricacies of code while mitigating the impact of domain-specific biases. Similarly, Wang et al. (Li et al. 2023) employed graph embeddings to encode Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), and Data Flow Graphs (DFGs), ensuring that syntactic and semantic features essential for vulnerability detection are preserved across domains. Their results highlight the utility of graph embeddings for achieving superior cross-project generalisation.

Domain adaptation techniques have been central to addressing domain shifts in CPVD. Nguyen et al. (Nguyen et al. 2020) proposed a dual-component framework that leverages adversarial learning and feature alignment to minimise domain divergence while preserving vulnerability-specific features. By aligning shared features across domains, their approach enhances the model's ability to detect vulnerabilities in diverse projects. Chen et al. (Liu et al. 2022) extended this concept with the CD-VulD framework, incorporating deep domain adaptation layers to align latent feature spaces and integrate structural and contextual features. These techniques underscore the importance of domain adaptation in CPVD and its role in improving model robustness.

Recent efforts have also explored hybrid models that combine multiple learning paradigms to enhance CPVD performance (Zhang et al. 2023). By integrating static analysis, graph-based learning, and representation learning, hybrid approaches address challenges such as class imbalance and variability in code structures. Additionally, emerging techniques like self-supervised and contrastive learning have demonstrated promise in pre-training models on large, unlabelled datasets, improving their ability to generalise and reducing false positives.

2.3 Research Gap and Our Contribution

Despite significant progress, CPVD still faces several key challenges. One major issue is aligning feature distributions between the source and target domains, especially when there are large differences in coding styles, structures, and libraries used across projects. Many existing domain adaptation methods struggle to handle the noise caused by inaccurate pseudo-labels, which can reduce the model's ability to correctly identify vulnerabilities. Additionally, most approaches rely solely on either graph-based models or transformer-based models, missing the potential benefits of combining both. Finally, class imbalance and the lack of labelled data in the target domain remain major obstacles to developing reliable and scalable CPVD solutions for real-world use..

Our approach addresses these gaps by introducing a unified framework that integrates domain-agnostic CodeBERT embeddings, iterative pseudo-labelling, and ensemble refinement to improve cross-project generalisation. By aligning source and target feature distributions through iterative adaptation, the proposed method reduces domain discrepancies and ensures that vulnerability-relevant features are retained. Unlike existing techniques, our framework combines transformer-derived embeddings, leveraging their complementary strengths to enhance recall and minimise false negatives. This comprehensive strategy aims to overcome the limitations of existing CPVD methods and provide a robust, scalable solution for real-world software security challenges.

3 Methodology

This section presents the step-by-step methodology of ZSVulD, the proposed framework for zero-shot, cross-project vulnerability detection, as illustrated in Figure 1. ZSVulD integrates domain-agnostic feature representation, iterative adaptation, and ensemble learning to address the domain shift challenges inherent in cross-project vulnerability detection. Each component is designed to align the source and target domains, facilitating robust generalisation without requiring labelled data from the target project.

The process begins with feature extraction using CodeBERT, where mean embeddings are generated to capture the source code's syntactic and semantic information. Subsequently, these embeddings are normalised to ensure consistency across domains. This is followed by the computation of class weights to mitigate class imbalance in the source dataset. ZSVulD then employs an iterative pseudo-labelling mechanism, where a neural network (NN) generates pseudo-labels for the unlabelled target data. These pseudo-labels are iteratively refined through ensemble learning with XGBoost, enhancing the model's generalisation capability.

The following subsections provide a detailed explanation of each component, outlining their specific roles and contributions within ZSVulD.

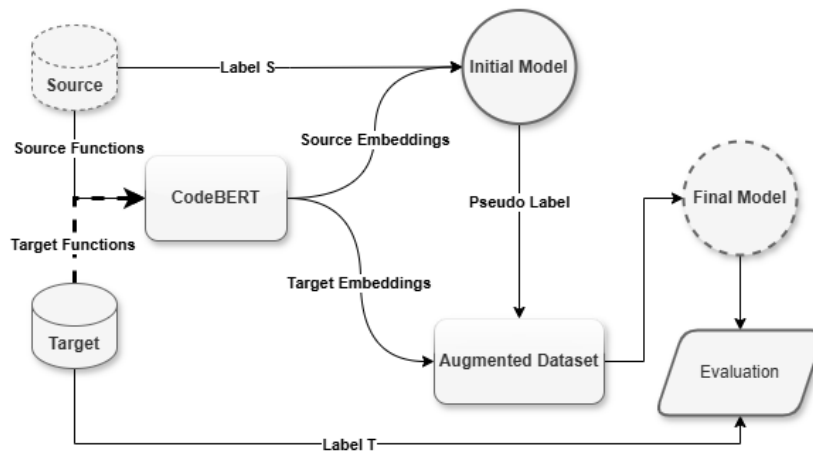


Figure 1. The framework of the proposed ZSVulD approach

3.1 Problem Formalisation

The task of zero-shot, cross-project vulnerability detection involves predicting the vulnerability labels of code functions in an unlabelled target dataset (D_T) using knowledge derived from a

labelled source dataset (D_S). This section formalises the problem, identifies the inherent challenges, and establishes the constraints under which the proposed framework operates.

3.1.1 Notations and Definitions

Let $D_S = (x_s^{(i)}, y_s^{(i)}) \mid i = 1, \dots, N_S$ represent the labelled source dataset, where $x_s^{(i)}$ denotes a function-level code snippet and $y_s^{(i)} \in 0,1$ is its corresponding vulnerability label. The target dataset, denoted as $D_T = x_t^{(j)} \mid j = 1, \dots, N_T$, consists of unlabelled code snippets. The objective is to learn a classifier f that assigns vulnerability labels to the target dataset:

$$\hat{y}_t^{(j)} = f(x_t^{(j)}) \quad (1)$$

3.1.2 Constraints and Challenges

- **Zero-Shot Setting:** The classifier must generalise to unseen projects without labelled target data.
- **Domain Shift:** Feature distributions between the source and target datasets differ significantly.
- **Class Imbalance:** Vulnerable functions are often underrepresented, affecting classifier performance.
- **Pseudo-Labeling Noise:** Pseudo-labelling can introduce errors that propagate across training iterations without labelled target data.

3.2 Code Representation Using CodeBERT

Code functions in the source (D_S) and target (D_T) datasets are converted into domain-agnostic embeddings using CodeBERT, a pre-trained transformer-based language model. CodeBERT's architecture enables it to capture both the syntactic and semantic features of code, making it an ideal choice for generating representations that generalise across projects (Feng et al. 2020). This section describes the embedding process, which involves tokenisation, chunking, pooling, and representation construction.

3.2.1 Tokenisation and Chunking

Each code function x in D_S or D_T is first tokenised into a sequence of tokens. CodeBERT processes token sequences with a maximum length of 512 tokens, imposing a constraint on how long functions are handled. For functions exceeding this length ($L > 512$), they are split into $\lfloor L/512 \rfloor$ chunks, where each chunk contains up to 512 tokens (Feng et al. 2020). These chunks are processed independently by CodeBERT to generate token embeddings.

For shorter functions ($L \leq 512$), the entire function is tokenised and processed in a single pass. Padding is applied as necessary to ensure compatibility with CodeBERT's input size requirements (Feng et al. 2020). This approach ensures that no part of the function is omitted during the embedding process, maintaining the completeness of the representation.

3.2.2 Mean Pooling for Unified Representations

To generate a single embedding for each function, the token embeddings produced by CodeBERT are aggregated across all chunks using mean pooling. This operation computes the average embedding across tokens and chunks (Feng et al. 2020):

$$h_{\text{mean}}(x) = \frac{1}{C \times 512} \sum_{l=1}^C \sum_{i=1}^{512} h_i^l, \quad (2)$$

where $h_{\text{mean}}(x) \in \mathbb{R}$ is the unified embedding for the function, $d = 768$ is the dimensionality of the CodeBERT embeddings, and $C = \lceil L/512 \rceil$ is the number of chunks for a function of length L (Feng et al. 2020). The mean-pooling operation ensures that the resulting embeddings are independent of function length, providing a consistent representation for both short and long code functions. This unified representation is compact yet retains the semantic and syntactic information encoded by CodeBERT.

3.2.3 Embedding Matrices for Source and Target Datasets

The embedding process is applied to all functions in the source and target datasets (Feng et al. 2020). For the source dataset D_S , the embeddings are represented as:

$$X_S = \left[h_{\text{mean}}(x_s^{(1)}), h_{\text{mean}}(x_s^{(2)}), \dots, h_{\text{mean}}(x_s^{(N_S)}) \right] \in \mathbb{R}^{(N_S \times d)}, \quad (3)$$

where each row corresponds to the embedding of a source function. Similarly, for the target dataset D_T , the embeddings are represented as:

$$X_T = \left[h_{\text{mean}}(x_t^{(1)}), h_{\text{mean}}(x_t^{(2)}), \dots, h_{\text{mean}}(x_t^{(N_T)}) \right] \in \mathbb{R}^{(N_T \times d)} \quad (4)$$

These embedding matrices serve as the input features for the subsequent learning stages, enabling the framework to handle source and target datasets uniformly.

3.2.4 Role of CodeBERT in Domain Bridging

The embeddings generated by CodeBERT are instrumental in reducing the domain gap between the source and target datasets. By mapping functions into a shared vector space, CodeBERT ensures that semantically similar functions from different projects are positioned closer to one another, irrespective of project-specific differences in coding style, library usage, or complexity (Feng et al. 2020).

CodeBERT is used in a frozen state throughout the embedding process, preserving the generalisable knowledge it has acquired during pre-training (Feng et al. 2020)(Devlin et al. 2019). This prevents overfitting to the source domain and ensures that the embeddings remain transferable to the target domain. The frozen embeddings capture domain-agnostic features, making them a robust foundation for the subsequent iterative adaptation and ensemble refinement stages.

3.3 Data Preprocessing

Data preprocessing is a critical step in ensuring the compatibility of the source and target datasets, as well as addressing inherent challenges such as feature distribution differences and class imbalance. The preprocessing steps involve the normalisation of embeddings to align feature distributions and the computation of class weights to mitigate the impact of imbalanced data in the source dataset.

3.3.1 Normalisation Across Domains

The embeddings extracted from CodeBERT for the source and target datasets may exhibit different distributions due to variations in coding style, function complexity, and project-specific factors. To ensure a consistent feature scale across both domains, embeddings are normalised using z-score normalisation. This process transforms each feature dimension to

have zero mean and unit variance, preventing certain features from dominating the model due to differences in scale. The normalisation process ensures that the source-trained model can generalise more effectively to the target domain.

3.3.2 Class Imbalance Handling

In real-world vulnerability datasets, the number of non-vulnerable functions ($y = 0$) often far exceeds the number of vulnerable functions ($y = 1$), leading to significant class imbalance. This imbalance can cause the model to bias its predictions toward the majority class, undermining its ability to detect vulnerabilities.

To address this issue, class weights are computed for the source dataset using the following formula:

$$w_c = \frac{N_S}{2 \cdot N_c}, \quad c \in \{0,1\} \quad (5)$$

where N_c is the number of samples in class c . These weights assign higher importance to the minority class during training, ensuring that both classes contribute equally to the loss function.

During training, the weighted binary cross-entropy loss is used to incorporate these class weights (Haque et al. 2024):

$$\mathcal{L}_{\text{weighted}} = -\frac{1}{N_S} \sum_{i=1}^{N_S} w_{y_s^{(i)}} \left[y_s^{(i)} \log p_s^{(i)} + (1 - y_s^{(i)}) \log(1 - p_s^{(i)}) \right] \quad (6)$$

where p_s^i is the model's predicted probability for the i -th source function. This approach prevents the model from being biased toward the majority class and enhances its ability to detect vulnerable functions.

3.4 Iterative Pseudo-Labelling and Neural Network Training

To adapt the source-trained model to the target domain, the framework employs an iterative pseudo-labelling approach. This method generates pseudo-labels for the target dataset, augments the training data with these labels, and refines the model iteratively to reduce the domain gap. This section describes the key steps in the iterative process, which include initial training on the source dataset, pseudo-label generation for the target dataset, and the re-training of the model with augmented data.

3.4.1 Initial Neural Network Training

The process begins with training a neural network (NN) on the source dataset $(\mathbf{X}_S, \mathbf{Y}_S)$. The NN is designed to map the source embeddings $\mathbf{X}_S \in \mathbb{R}^{N_S \times d}$ to binary vulnerability predictions $y_s^{(i)} \in \{0,1\}$. The training process minimises a weighted binary cross-entropy loss (Haque et al. 2024):

$$\mathcal{L}_{\text{weighted}} = -\frac{1}{N_S} \sum_{i=1}^{N_S} w_{y_s^{(i)}} \left[y_s^{(i)} \log p_s^{(i)} + (1 - y_s^{(i)}) \log(1 - p_s^{(i)}) \right] \quad (7)$$

where $p_s^{(i)}$ is the model's predicted probability for the i -th source function, and $w_{y_s^{(i)}}$ represents the class weight for the function's label $y_s^{(i)}$. The NN outputs probabilities for both source and target embeddings:

$$p_s^{(i)} = f_{\text{NN}}(x_s^{(i)}), \quad p_t^{(j)} = f_{\text{NN}}(x_t^{(j)}) \quad (8)$$

where f_{NN} represents the learned mapping function of the NN . After training, the model achieves a baseline capability for vulnerability prediction in the source domain.

3.4.2 Pseudo-Label Generation

Using the trained NN , pseudo-labels are generated for the target dataset. For each target function $x_t^{(j)} \in D_T$, the model predicts a probability $p_t^{(j)}$, which is the threshold to assign a binary pseudo-label:

$$\hat{y}_t^{(j)} = \begin{cases} 1 & \text{if } p_t^{(j)} \geq \tau, \\ 0 & \text{otherwise.} \end{cases} \quad (9)$$

Here, τ is a predefined threshold (commonly set to 0.5) that balances sensitivity and specificity in the pseudo-labelling process (Haque et al. 2024).

While pseudo-labels may initially be noisy due to the domain gap, iterative refinement and ensemble techniques (described in subsequent sections) help mitigate the impact of these errors.

3.4.3 Data Augmentation

The generated pseudo-labels for the target dataset are combined with the labelled source data to create an augmented training set:

$$D_{\text{augmented}} = (\mathbf{X}_S \cup \mathbf{X}_T, \mathbf{Y}_S \cup \widehat{\mathbf{Y}}_T), \quad (10)$$

where $\widehat{\mathbf{Y}}_T = [\hat{y}_t^{(1)}, \hat{y}_t^{(2)}, \dots, \hat{y}_t^{(N_T)}]$ represents the pseudo-labels for the target dataset. This augmentation enables the NN to learn features that are more representative of the target domain.

3.4.4 Iterative Refinement

The augmented dataset is used to re-train the NN , progressively aligning its decision boundaries with the target domain. During each iteration i :

1. The NN is fine-tuned on $D_{\text{augmented}}$, using the weighted binary cross-entropy loss to account for class imbalance.
2. Updated pseudo-labels are generated for the target dataset based on the fine-tuned model:

$$\hat{y}_t^{(j)}(i) = \begin{cases} 1 & \text{if } p_t^{(j)} \geq \tau, \\ 0 & \text{otherwise.} \end{cases} \quad (11)$$

3. The augmented dataset is updated with the new pseudo-labels, and the process is repeated for N iterations.

This iterative refinement process reduces the noise in pseudo-labels over time, allowing the model to better generalise to the target domain. The number of iterations N is chosen empirically, balancing computational cost with the need for accurate adaptation (Haque et al. 2024).

3.4.5 Pseudo-Labeling Noise and Mitigation

While pseudo-labelling introduces noise, particularly in early iterations, the framework mitigates its impact through:

1. **Weighted Learning:** Source data retains higher confidence due to its ground-truth labels, guiding the model during the early stages of adaptation.
2. **Ensemble Refinement:** Predictions from multiple iterations are aggregated to stabilise pseudo-label quality, as described in Section 3.5.

This iterative pseudo-labelling approach forms the foundation of the domain adaptation process, progressively aligning the source-trained model with the target domain by incorporating pseudo-labelled data and iterative refinement.

3.5 XGBoost Refinement and Ensemble Learning

While the iterative pseudo-labelling process gradually adapts the neural network (NN) to the target domain, predictions in early iterations can be noisy due to domain discrepancies and the absence of labelled target data. To address this, the framework integrates XGBoost as a complementary model for residual learning and employs ensemble techniques to stabilise pseudo-label predictions across iterations.

3.5.1 Rationale for XGBoost Refinement

XGBoost is a gradient-boosted decision tree algorithm known for its ability to capture complex feature interactions and residual patterns that may be overlooked by deep neural networks. In this framework, XGBoost is employed after each iteration of pseudo-labelling to refine predictions for the target dataset. By leveraging the pseudo-labelled augmented dataset, XGBoost complements the NN by learning from both correctly labelled and mislabelled samples, providing additional robustness to the framework.

The pseudo-labelled augmented dataset at iteration i is denoted as:

$$D_{augmented}^{(i)} = (X_S \cup X_T, Y_S \cup \hat{Y}_T^{(i)}), \quad (12)$$

Where $\hat{Y}_T^{(i)} = [\hat{y}_t^{(1)}(i), \hat{y}_t^{(2)}(i), \dots, \hat{y}_t^{(N_T)}(i)]$ represents the updated pseudo-labels for the target dataset.

XGBoost is trained on this augmented dataset to predict the vulnerability labels of target functions. The training process incorporates pseudo-label confidence through sample weights (Chen and Guestrin 2016).

3.5.2 Sample Weighting for Pseudo-Labels

The framework computes sample weights based on the NN's predicted probabilities to account for the uncertainty in pseudo-labels. For a pseudo-labelled target function $x_t^{(j)}$, the sample weight is defined as:

$$w_{sample}^{(j)} = \begin{cases} p_t^{(j)}(i) & \text{if } \hat{Y}_T^{(j)}(i) = 1, \\ 1 - p_t^{(j)}(i) & \text{if } \hat{Y}_T^{(j)}(i) = 0. \end{cases} \quad (13)$$

Here, $p_t^{(j)}(i)$ is the NN's predicted probability for $x_t^{(j)}$ at iteration i . These weights ensure that high-confidence pseudo-labels have a greater influence during XGBoost training (Arazo et al. 2019), while low-confidence labels contribute less. The final training dataset for XGBoost is:

$$(X_S \cup X_T, Y_S \cup \hat{Y}_T^{(i)}, W_{sample}), \quad (14)$$

where W_{sample} contains the computed sample weights.

3.5.3 Ensemble Prediction Across Iterations

After training, XGBoost predicts vulnerability labels for the target dataset. The predictions from XGBoost and the NN are aggregated over multiple iterations to form an ensemble prediction. For each target function $x_t^{(j)}$, the ensemble prediction at iteration i is:

$$\hat{y}_t^{(j)}(i) = \alpha \cdot f_{NN}^{(i)}(x_t^{(j)}) + (1 - \alpha) \cdot f_{XGB}^{(i)}(x_t^{(j)}), \quad (15)$$

where $f_{NN}^{(i)}$ and $f_{XGB}^{(i)}$ are the outputs of the NN and XGBoost models, respectively, at iteration i , and $\alpha \in [0,1]$ is a weighting parameter balancing the contributions of the two models (Haque et al. 2024). Over N iterations, the ensemble aggregates predictions for each function:

$$\text{Ensemble}_t = \frac{1}{N} \sum_{i=1}^N \hat{y}_t^{(1)}(i) \quad (16)$$

The final prediction is obtained by thresholding the ensemble output:

$$\hat{y}_t^{final} = \begin{cases} 1 & \text{if } \text{Ensemble}_t > 0.5, \\ 0 & \text{otherwise.} \end{cases} \quad (17)$$

3.5.4 Noise Mitigation Through Ensembles

The ensemble approach mitigates the noise introduced by pseudo-labelling in early iterations. By averaging predictions across multiple iterations and models, the framework reduces the influence of outlier predictions and stabilises the final outputs. This multi-model ensemble provides a robust mechanism for aligning the decision boundaries of the NN and XGBoost with the target domain.

3.6 Final Predictions

After completing N iterations of pseudo-labelling, neural network refinement, and XGBoost learning, the framework combines predictions from multiple iterations and models to produce the final vulnerability predictions for the target dataset. This section outlines the aggregation of predictions and the final decision-making process.

3.6.1 Aggregation of Predictions

The iterative nature of the framework generates predictions for the target dataset at each iteration i . For a target function $x_t^{(j)}$, the predictions from the neural network $(f_{NN}^{(i)}(x_t^{(j)}))$ and XGBoost $(f_{XGB}^{(i)}(x_t^{(j)}))$ are combined using an ensemble approach. Over N iterations, the ensemble output for $x_t^{(j)}$ is computed as:

$$\text{Ensemble}_t^{(j)} = \frac{1}{N} \sum_{i=1}^N [\alpha \cdot f_{NN}^{(i)}(x_t^{(j)}) + (1 - \alpha) \cdot f_{XGB}^{(i)}(x_t^{(j)})], \quad (18)$$

where $\alpha \in [0, 1]$ balances the contributions of the neural network and XGBoost models.

The final prediction for $x_t^{(j)}$ is obtained by thresholding the ensemble output:

$$\hat{y}_t^{(j)} = \begin{cases} 1 & \text{if } \text{Ensemble}_t^{(j)} > 0.5, \\ 0 & \text{otherwise.} \end{cases} \quad (19)$$

This approach ensures that the final predictions incorporate the strengths of both models while reducing the impact of noisy pseudo-labels from early iterations.

3.6.2 Final Outputs

The outputs of the framework are the predicted vulnerability labels for the target dataset:

$$\widehat{Y}_T = [\hat{y}_t^{(1)}, \hat{y}_t^{(2)}, \hat{y}_t^{(N_T)}] \quad (20)$$

These predictions reflect the neural network's and XGBoost models' combined insights, stabilised over multiple iterations. By leveraging ensemble techniques, the framework provides robust and accurate predictions for cross-project vulnerability detection, even in the challenging zero-shot setting.

Algorithm 1: Algorithm for the Proposed ZSVulD Approach

Input:

- Source Dataset: D_S (labelled)
- Target Dataset: D_T (unlabelled)
- N : Number of iterations
- NN : Neural network model
- XGB : XGBoost model

Output:

- $Y_{T_{Final}}$: Final predictions for D_T
- Performance metrics: Recall, Precision, F1-score, F2-score

Steps:

1. Embedding Generation:
 - Generate embeddings for source dataset (D_S) and target dataset (D_T) using CodeBERT:
 - Compute mean embeddings for each function/code function in D_S and D_T .
 - Represent the generated embeddings as X_S (source) and X_T (target).
 - Extract corresponding labels Y_S from D_S . Target labels Y_T are unlabeled.
2. Preprocessing:
 - Normalize X_S and X_T using a standard scaler.
 - Compute class weights W for Y_S to handle class imbalance.
3. Initialization:
 - Initialize EnsemblePredictions as a zero vector of length X_T .
4. Iterative Training:

For $i = 1$ to N (number of boosting iterations):

 - 4.1 Neural Network Training:
 - Train NN on X_S and Y_S with early stopping using validation loss.
 - 4.2 Pseudo-Label Generation:
 - Generate pseudo-labels Y'_T for X_T using NN :
 $Y'_T = \text{sigmoid}(NN(X_T)) > 0.5$
 - 4.3 Augment Training Data:
 - Augment datasets by concatenating source and pseudo-labelled target data:
 $X_{Aug} = \text{concatenate}(X_S, X_T)$
 $Y_{Aug} = \text{concatenate}(Y_S, Y'_T)$
 - Fine-tune NN on X_{Aug} and Y_{Aug} .
 - 4.4 Sample Weighting for XGBoost:
 - Compute sample weights W_{Sample} :
 If $y = 1$: $W_{Sample}(y) = Y_{Aug}$
 If $y = 0$: $W_{Sample}(y) = 1 - Y_{Aug}$
 - 4.5 XGBoost Training:
 - Train XGBoost on X_S and Y_S using W_{Sample} with early stopping.
 - 4.6 Prediction and Ensemble Update:
 - Predict target labels for X_T using XGBoost:

$$Y_{T(i)} = XGB(X_T)$$

- Update ensemble predictions:

$$EnsemblePredictions = EnsemblePredictions + Y_{T(i)}$$

5. Final Prediction:

- Compute final predictions for Y_T :

$$Y_{T_{Final}} = \left(\frac{EnsemblePredictions}{N} \right) > 0.5$$

4 Experimental Setup

The experimental setup provides details of the research questions, datasets, evaluation metrics, and implementation specifics used to validate the proposed framework for zero-shot, cross-project vulnerability detection.

4.1 Research Questions

To evaluate the proposed ZSVulD approach, we have designed the following research questions (RQs):

RQ1: How does fine-tuned CodeBERT perform in cross-project vulnerability detection?

RQ2: How effectively does the proposed framework generalise to unseen target projects in a zero-shot setting, where no labelled target data is available?

RQ3: How well do CodeBERT embeddings capture vulnerability-relevant features, and how effectively does the framework align source and target feature distributions?

RQ4: How does the proposed framework compare to state-of-the-art cross-project vulnerability detection methods in terms of recall, precision, F1-score, and F2-score?

4.2 Datasets

The experiments utilise two widely recognised datasets, Devign (Zhou et al.) and REVEAL (Chakraborty et al. 2022), which consists of labelled C code functions from a range of software projects. Both datasets include examples of vulnerable and non-vulnerable functions, making them suitable for training and evaluating vulnerability detection models. The combined dataset comprises a total of 30,260 code samples, with 14,700 labelled as vulnerable and 15,560 as non-vulnerable. Devign focuses on real-world vulnerability scenarios extracted from large-scale open-source projects such as FFmpeg and Qemu, providing a diverse set of challenging examples with varying coding styles and complexities. REVEAL, on the other hand, includes data from projects such as Debian and Chrome, offering additional diversity in terms of library usage and structural intricacies. These project-specific variations ensure that the datasets represent realistic and heterogeneous environments, ideal for assessing the cross-project generalisation capability of the proposed framework. For a fair comparison, we used the same dataset from the paper (Li et al. 2023).

Further details and statistical breakdowns for these datasets are provided in Table 1, illustrating their scale and the distribution of vulnerable and non-vulnerable samples across projects.

Table 1. Experimental Dataset Descriptions

Project	Dataset	Vulnerable	Non-Vulnerable	Total
Devign	FFmpeg	4981	5016	9997
	Qemu	7479	7980	15459
REVEAL	Debian	1415	1653	3068
	Chrome	825	911	1768
Total		14700	15560	30260

4.3 Experimental details

The proposed framework processes 768-dimensional mean embeddings generated by CodeBERT, ensuring a domain-agnostic representation of source code. These embeddings are normalised to maintain consistency between the source and target datasets, preventing discrepancies that could arise due to differences in coding styles and project-specific features.

4.3.1 Fine-Tuning CodeBERT

For baseline evaluation, CodeBERT was fine-tuned on each source dataset before being applied to cross-project scenarios. The fine-tuning process was conducted with the Adam optimiser at a learning rate of 1×10^{-5} , using a 70:15:15 train-validation-test split. The model was trained for a maximum of 10 epochs, with early stopping based on validation F1-score and patience of 2 epochs to prevent overfitting.

4.3.2 Neural Network Configuration

The primary model is a deep neural network (NN) designed to capture non-linear relationships in the CodeBERT embeddings. The NN consists of:

- Four hidden layers with 128, 64, 32, and 32 neurons, respectively, each activated using the ReLU function.
- Dropout layers with a rate of 0.2 are applied after each hidden layer to improve generalisation and mitigate overfitting.
- A sigmoid activation function in the output layer to produce probability scores for vulnerability predictions.
- The Adam optimiser with a learning rate of 1×10^{-5} , optimised using weighted binary cross-entropy loss to handle class imbalance.

4.3.3 XGBoost Model for Refinement

To further enhance the prediction quality, the framework integrates an XGBoost classifier for residual learning. The XGBoost model consists of

- 100 decision trees, each with a maximum depth of 3 to limit overfitting.
- A learning rate of 0.1 for gradient-boosted updates.
- Sample weights based on pseudo-label confidence scores to guide learning in the absence of labelled target data.

The predictions from the NN and XGBoost models are combined iteratively, and ensemble averaging is employed over multiple iterations to stabilise the final outputs.

4.3.4 Comparative Analysis and Baseline Models

For comparative analysis, the performance of the proposed framework was evaluated alongside existing methods from the literature, including VulGDA (Li et al. 2023), VulDPecker (Li et al. 2018), and SCDAN (Li et al. 2022). The results for these baseline methods we obtained from this paper (Li et al. 2023), which reported their performance on the same datasets used in this study. This approach ensures a fair and consistent comparison under identical experimental settings.

4.3.5 Implementation and Hardware Setup

The experiments were conducted on a high-performance computing setup, equipped with an Intel Core i9-11900K CPU, 64 GB of RAM, and an NVIDIA RTX 4090 GPU with 24 GB of memory. The implementation was carried out using Python, leveraging TensorFlow for deep learning model training, Scikit-learn for preprocessing tasks, and XGBoost for gradient-boosted decision trees. To prevent overfitting, both the neural network (NN) and XGBoost models incorporated early stopping, ensuring stable and reliable predictions throughout the training process.

4.4 Evaluation Metrics

To evaluate the proposed zero-shot cross-project vulnerability detection framework, we use Recall, Precision, F1-score, and F2-score. These metrics are particularly important for binary classification tasks involving imbalanced datasets, such as vulnerability detection, where the vulnerable class is often underrepresented.

Recall:

Recall is critical in vulnerability detection as it measures the model's ability to identify actual positive samples (vulnerable functions). Missing a vulnerability (false negative) could lead to significant security risks. Recall is calculated as:

$$Recall = \frac{TP}{TP + FN}$$

where TP represents True Positives (correctly identified vulnerabilities) and FN represents False Negatives (missed vulnerabilities).

Precision:

Precision evaluates the proportion of correctly identified vulnerabilities among all predicted vulnerabilities. High precision ensures the model does not overwhelm developers with false positives, which could dilute the effectiveness of vulnerability alerts (Goutte and Gaussier 2005). Precision is calculated as:

$$Precision = \frac{TP}{TP + FP}$$

where FP represents False Positives (non-vulnerable functions misclassified as vulnerable).

F1-Score:

The F1-score provides a harmonic mean of precision and recall, balancing the trade-off between avoiding false positives and capturing all vulnerabilities (Goutte and Gaussier 2005). It is particularly useful when both metrics are equally important :

$$F1 - Score = 2 * \frac{Precision \times Recall}{Precision + Recall}$$

F2-Score:

The F2-score, a variant of the F1-score, gives higher weight to recall, making it especially relevant in vulnerability detection, where the priority is to identify as many vulnerabilities as possible, even if some false positives occur (Zhou et al. 2021). It is calculated as:

$$F2 - Score = (1 + 2^2) * \frac{Precision \times Recall}{(2^2 * Precision + Recall)}$$

These metrics ensure a balanced evaluation, particularly in the presence of class imbalance.

5 Results and Discussion

5.1 Answer to RQ1: How does fine-tuned CodeBERT perform in cross-project vulnerability detection?

To evaluate the effectiveness of fine-tuned CodeBERT in cross-project vulnerability detection, we conducted experiments where the model was first fine-tuned on each source dataset before being applied in a zero-shot setting to an unseen target dataset. The goal was to determine whether fine-tuning CodeBERT on project-specific data improves its ability to generalise across different software repositories. The results, summarised in Table 2, indicate that while fine-tuning enhances performance within the source dataset, it does not significantly improve generalisation in cross-project scenarios.

Table 2. Performance of Fine-Tuned CodeBERT in Cross-Project Scenario

Source	Target	Recall	Precision	F1-Score	F2-Score
Devign	Reveal	0.66	0.58	0.62	0.64
Reveal+Qemu	FFmpeg	0.66	0.54	0.59	0.63
Devign+Chrome	Debian	0.71	0.52	0.61	0.66

The results highlight a clear trend: while recall remains relatively high (0.66 - 0.71), precision is significantly lower (0.52 - 0.58). This suggests that the fine-tuned CodeBERT model effectively detects vulnerable functions but struggles with false positives, reducing its reliability in real-world security applications. The F2-score consistently surpasses the F1-score, reinforcing the model's bias towards recall rather than precision. This aligns with the nature of vulnerability detection, where prioritising recall is essential to minimise the risk of undetected vulnerabilities.

The limitations of fine-tuning CodeBERT for cross-project settings stem from overfitting to project-specific characteristics. Since large language models (LLMs) like CodeBERT are pre-trained on extensive datasets, fine-tuning on a single project dataset over-specialises the model to the stylistic, syntactic, and structural patterns present in that dataset. This reduces its ability to generalise when applied to a different project, as its learnt representations become highly project-specific rather than domain-agnostic.

Additionally, the domain shift challenge further limits the model’s generalisation. Each project varies in coding style, function structures, library dependencies, and security vulnerability patterns, making it difficult for fine-tuned CodeBERT to retain universal representations applicable across multiple domains. This is evident from the F1 and F2 scores, which show that fine-tuning does not significantly enhance cross-project performance, despite refining within-project accuracy.

Figure 2 illustrates the impact of these limitations by comparing the performance of fine-tuned CodeBERT across different cross-project scenarios. The low precision and high recall confirm the model’s tendency to classify more functions as vulnerable, increasing the false positive rate.

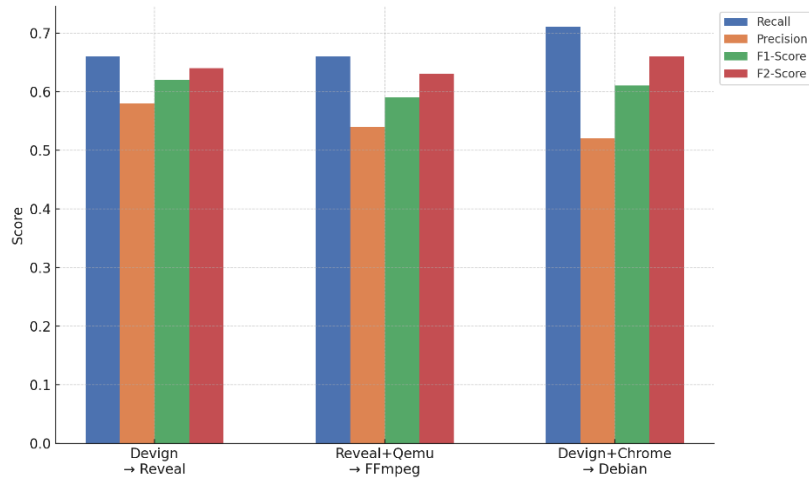


Figure 2. Performance of Fine-Tuned CodeBERT in Cross-Project Settings

These findings demonstrate that fine-tuning CodeBERT alone is insufficient for cross-project vulnerability detection. Unlike domain-agnostic embeddings, which maintain a broader representation of code features, fine-tuned models become overly tailored to specific training datasets, making them ineffective for zero-shot adaptation. This highlights the need for alternative strategies, such as feature alignment and iterative pseudo-labelling, as integrated into our proposed framework, which enables superior cross-project generalisation without overfitting to project-specific data.

5.2 Answer to RQ2: How effectively does the proposed framework generalise to unseen target projects in a zero-shot setting, where no labelled target data is available?

The proposed framework demonstrates strong generalisation capabilities in zero-shot cross-project vulnerability detection, as evidenced by its performance across three distinct cross-

project scenarios: Devign $\rightarrow$ Reveal, Reveal+Qemu $\rightarrow$ FFmpeg, and Devign+Chrome $\rightarrow$ Debian. The results, presented in Table 3, indicate consistently high recall values, particularly 0.87 and 0.85 in the Devign $\rightarrow$ Reveal and Devign+Chrome $\rightarrow$ Debian settings, respectively. These recall scores highlight the framework’s effectiveness in identifying vulnerabilities across previously unseen projects, minimising false negatives—a critical aspect of security vulnerability detection.

The F2 scores of 0.77 and 0.76 reinforce the emphasis on recall, reflecting the framework’s prioritisation of identifying vulnerabilities over precision. Even in the Reveal+Qemu $\rightarrow$ FFmpeg scenario, where the recall is slightly lower at 0.73, the framework still maintains a strong capacity for cross-project adaptation. The F1 scores, ranging from 0.63 to 0.66, further indicate that the framework achieves a stable balance between recall and precision across different target projects.

Table 3. Performance of the proposed ZSVulD approach in Cross-Project Scenario

Source	Target	Recall	Precision	F1-Score	F2-Score
Devign	Reveal	0.87	0.52	0.65	0.77
Reveal+Qemu	FFmpeg	0.73	0.53	0.63	0.69
Devign+Chrome	Debian	0.85	0.54	0.66	0.76

Figure 3 illustrates the impact of the framework’s generalisation capability, showcasing high recall and F2-score performance across all three scenarios. The ability to align feature distributions effectively enables knowledge transfer from source to target projects, mitigating domain discrepancies and enhancing vulnerability detection in unseen code functions.

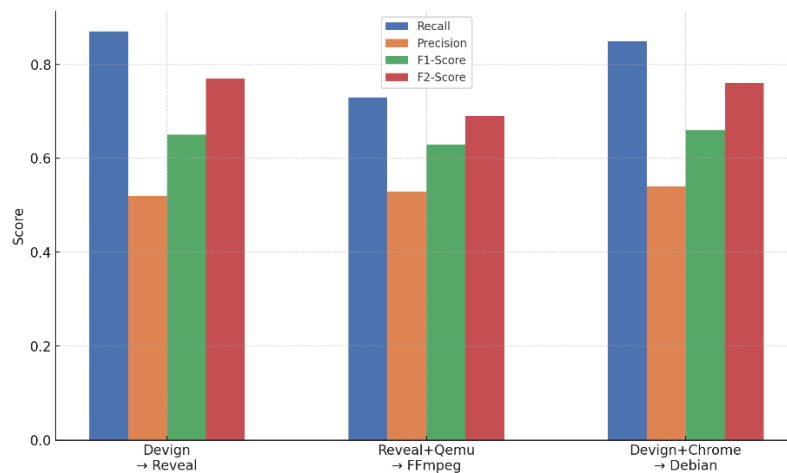


Figure 3. Performance the Proposed ZSVulD Approach in Cross-Project Scenario

These findings confirm that the proposed framework effectively generalises to unseen target projects in a zero-shot setting, successfully transferring learned representations without requiring labelled target data. Its ability to maintain high recall and F2 scores across diverse software repositories demonstrates its robustness in real-world vulnerability detection applications, where domain shifts and lack of labelled data are significant challenges.

5.3 Answer to RQ3: How well do CodeBERT embeddings capture vulnerability-relevant features, and how effectively does the framework align source and target feature distributions?

The alignment of feature distributions between the source and target domains is a key component of the proposed framework, addressing the domain gap inherent in zero-shot cross-project vulnerability detection. Figures 4-6 showcase the evolution of feature alignment before and after adaptation in three distinct cross-project scenarios. Initially, as seen in the "Before" plots, the source and target feature distributions exhibit clear separations, highlighting the significant differences in coding styles, library usage, and complexity between projects. These misalignments pose challenges to effective knowledge transfer, as source-trained models struggle to generalise to target domains.

After employing the proposed iterative pseudo-labelling and ensemble refinement mechanisms, the "After" plots illustrate a remarkable reduction in feature distribution divergence. The t-SNE visualisations reveal substantial overlaps between the source and target embeddings, indicating successful feature alignment. This alignment is achieved through domain-agnostic embeddings generated by CodeBERT and the iterative refinement process that minimises domain-specific biases while retaining vulnerability-relevant features. The closer proximity of the embeddings in the "After" plots demonstrates the framework's ability to bridge the domain gap, enabling semantically similar code functions from different domains to occupy similar regions in the feature space.

This aligned feature space significantly enhances the framework's generalisation capability by facilitating effective knowledge transfer across domains. The t-SNE visualisations confirm that the proposed approach reduces domain discrepancies, making it a robust solution for zero-shot cross-project vulnerability detection. These results emphasise the importance of feature alignment in achieving reliable predictions across diverse datasets.

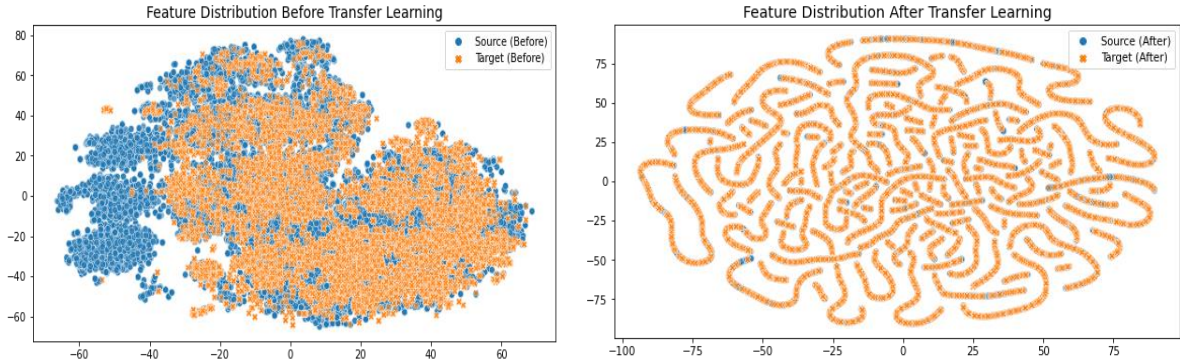


Figure 4. Feature Alignment: Devign → Reveal

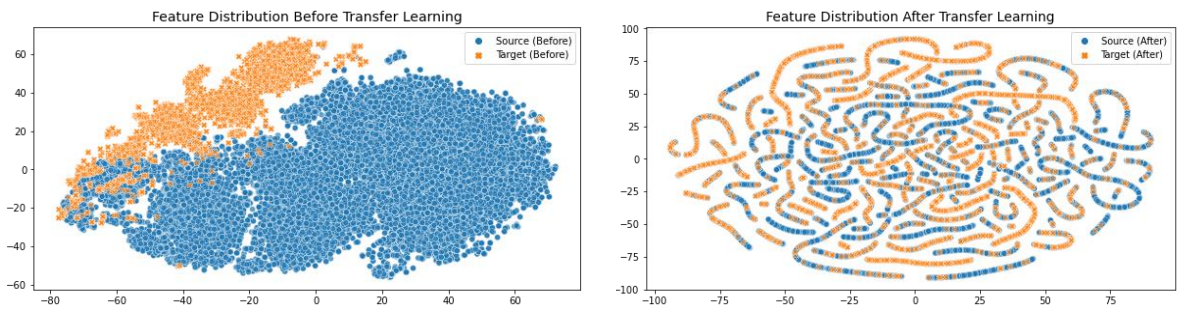


Figure 5. Feature Alignment: Reveal+Qemu → FFmpeg

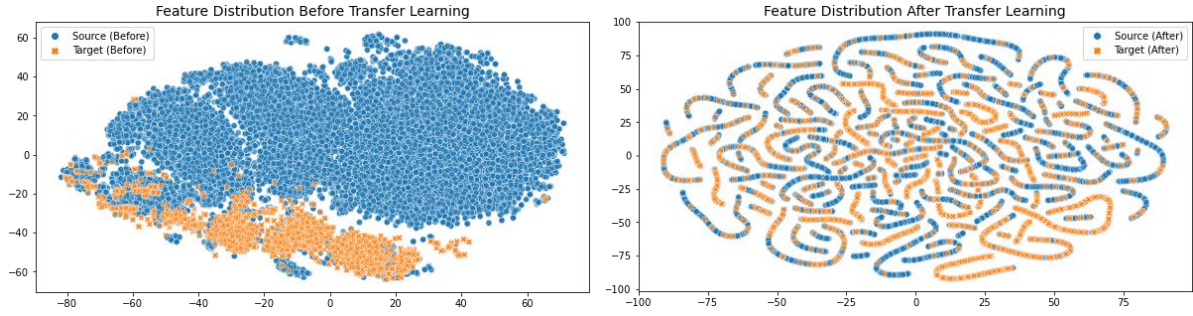


Figure 6. Feature Alignment: Devign+Chrome → Debian

5.4 Answer to the RQ4: How does the proposed framework compare to state-of-the-art cross-project vulnerability detection methods in terms of recall, precision, F1-score, and F2-score?

The comparative analysis, presented in Table 4, demonstrates that the proposed framework outperforms state-of-the-art cross-project vulnerability detection methods across multiple scenarios. The evaluation, conducted against VulGDA (Li et al. 2023), VulDPecker (Li et al. 2018), SCDAN (Li et al. 2022), and CodeBERT (Feng et al. 2020), highlights improvements in recall, F1-score, and F2-score, which are crucial for effective vulnerability detection.

In the Reveal+Qemu → FFmpeg scenario, the proposed framework achieves a recall of 0.73, outperforming VulGDA (0.61), VulDPecker (0.59), SCDAN (0.67), and CodeBERT (0.66). A similar trend is observed in the Devign → Reveal setting, where the framework attains a recall of 0.87, significantly surpassing VulGDA (0.52), VulDPecker (0.61), SCDAN (0.62), and CodeBERT (0.66). The Devign+Chrome → Debian case further confirms this pattern, with the proposed approach achieving a recall of 0.85, exceeding CodeBERT’s recall of 0.71. These results indicate the framework’s ability to identify vulnerabilities with fewer false negatives, a critical requirement in security-sensitive applications.

While the precision of the proposed framework is slightly lower, ranging between 0.52 and 0.54, this is consistent with the trade-offs commonly observed in high-recall models for vulnerability detection. The F1 scores remain competitive, with values of 0.63 in Reveal+Qemu → FFmpeg, 0.65 in Devign → Reveal, and 0.66 in Devign+Chrome → Debian, matching or exceeding those of existing methods. The F2 scores, which emphasise recall, further highlight the framework’s advantages, reaching 0.69, 0.77, and 0.76 across the respective scenarios—the highest among all evaluated methods.

Table 4. The performance of the proposed framework compared to the state of the art methods

Source	Target	Method	Recall	Precision	F1-score	F2-score
Reveal+ Qemu	FFmpeg	VulGDA (Li et al. 2023)	0.61	0.65	0.63	0.62
		VulDPecker (Li et al. 2018)	0.59	0.51	0.55	0.57
		SCDAN (Li et al. 2022)	0.67	0.60	0.63	0.65
		CodeBERT (Feng et al. 2020)	0.66	0.58	0.62	0.64
		Proposed	0.73	0.53	0.63	0.69

Devign	Reveal	VulGDA (Li et al. 2023)	0.52	0.67	0.59	0.54
		VulDPecker (Li et al. 2018)	0.61	0.54	0.57	0.59
		SCDAN (Li et al. 2022)	0.62	0.56	0.59	0.61
		CodeBERT (Feng et al. 2020)	0.66	0.54	0.59	0.63
		Proposed	0.87	0.52	0.65	0.77
Devign+Chrome	Debian	VulGDA (Li et al. 2023)	<i>Not applicable</i>	<i>Not applicable</i>	<i>Not applicable</i>	<i>Not applicable</i>
		VulDPecker (Li et al. 2018)	<i>Not applicable</i>	<i>Not applicable</i>	<i>Not applicable</i>	<i>Not applicable</i>
		SCDAN (Li et al. 2022)	<i>Not applicable</i>	<i>Not applicable</i>	<i>Not applicable</i>	<i>Not applicable</i>
		CodeBERT (Feng et al. 2020)	0.71	0.52	0.61	0.66
		Proposed	0.85	0.54	0.66	0.76

These findings confirm that the proposed framework outperforms state-of-the-art approaches in zero-shot cross-project vulnerability detection, particularly in terms of recall and F2-score, making it highly suitable for real-world security applications where missing vulnerabilities can have severe consequences.

6 Threats to Validity

While the proposed framework demonstrates significant promise for zero-shot, cross-project vulnerability detection, potential threats to validity must be acknowledged. These are grouped into internal, and external validity for clarity.

6.1 Internal Validity

The reliability of the experimental results may be influenced by the choice of representations and evaluation methods. The reliance on pre-trained embeddings from CodeBERT, while effective for capturing semantic and syntactic properties of code, may introduce biases inherited from the original training corpus, potentially favouring certain coding styles or structures. Furthermore, the iterative pseudo-labelling mechanism, though stabilised through ensemble refinement, may propagate errors introduced by noisy pseudo-labels in its early stages. The evaluation metrics like, recall, precision, F1-score, and F2-score provide a comprehensive view of the framework's performance, with an emphasis on recall and F2-score due to their importance in vulnerability detection. However, the prioritisation of high recall may lead to increased false positives, which, while acceptable in security contexts, could burden developers with unnecessary alerts. Additional studies on balancing recall and precision in real-world applications would provide a more nuanced understanding of the trade-offs involved.

6.2 External Validity

The datasets used in this study, Devign and REVEAL, are publicly available and widely recognised in vulnerability detection research, ensuring comparability with prior works. However, these datasets predominantly contain C functions and may not fully capture the diversity of programming languages, coding styles, or domain-specific characteristics found in

real-world scenarios. The generalisability of the framework to other programming languages, such as Java or Python, remains to be explored. Expanding the evaluation to datasets from additional domains and languages would further validate the proposed approach. To mitigate these threats, we selected datasets representative of real-world vulnerabilities and employed t-SNE visualisations to validate feature alignment across source and target domains. Nonetheless, the results may still reflect dataset-specific characteristics, highlighting the need for broader evaluations across varied contexts.

7 Conclusion and Future Work

This paper presents a novel framework for zero-shot, cross-project vulnerability detection in source code, addressing key challenges in domain adaptation and generalisation. By leveraging domain-agnostic CodeBERT embeddings, the framework captures both syntactic and semantic features of source code, enabling robust knowledge transfer without the need for fine-tuning. The iterative pseudo-labelling mechanism, integrated with ensemble refinement using neural networks and XGBoost, progressively aligns source and target feature distributions. Evaluations on real-life datasets, Devign and REVEAL, demonstrate the framework's effectiveness in reducing domain discrepancies, achieving high recall and balanced F1 and F2 scores, and outperforming state-of-the-art methods in several scenarios. The results highlight the framework's potential applications, including its integration into CI/CD pipelines for real-time security analysis and its use in large-scale open-source software projects to enhance code quality in the absence of labelled target data.

Despite its strong performance, several areas remain for future research. Integrating Code Property Graph features with CodeBERT embeddings could provide a richer representation by capturing additional structural and contextual relationships in the code. Hybrid approaches combining graph-based techniques and transformer-derived embeddings offer another promising direction for detecting complex vulnerabilities. Furthermore, refining pseudo-labelling strategies through confidence-based filtering or noise-aware mechanisms could further stabilise predictions and enhance model robustness.

Data Availability

The dataset used in this research is publicly available. For the convenience of readers, we have gathered all relevant datasets in one place to make them easily accessible. You can find the dataset here: [GitHub Repository](#).

Declarations

Conflict of interest The authors declare that they have no Conflict of interest.

References

Akter MS, Shahriar H, Bhuiya ZA (2023) Automated Vulnerability Detection in Source Code Using Quantum Natural Language Processing. *Communications in Computer and Information Science* 1768 CCIS:83–102. https://doi.org/10.1007/978-981-99-0272-9_6

- Arazo E, Ortego Di, Albert P, et al (2019) Pseudo-Labeling and Confirmation Bias in Deep Semi-Supervised Learning. Proceedings of the International Joint Conference on Neural Networks. <https://doi.org/10.1109/IJCNN48605.2020.9207304>
- Cai Z, Cai Y, Chen X, et al (2024) CSVD-TF: Cross-project software vulnerability detection with TrAdaBoost by fusing expert metrics and semantic metrics. Journal of Systems and Software 213:112038. <https://doi.org/10.1016/j.jss.2024.112038>
- Chakraborty S, Krishna R, Ding Y, Ray B (2022) Deep Learning Based Vulnerability Detection: Are We There Yet? IEEE Transactions on Software Engineering 48:3280–3296. <https://doi.org/10.1109/TSE.2021.3087402>
- Chen T, Guestrin C (2016) XGBoost: A scalable tree boosting system. Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining 13-17-August-2016:785–794. https://doi.org/10.1145/2939672.2939785/SUPPL_FILE/KDD2016_CHEN_BOOSTING_SYSTEM_01-ACM.MP4
- Chen Y, Ding Z, Alowain L, et al (2023) DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection. ACM International Conference Proceeding Series 15:654–668. <https://doi.org/10.1145/3607199.3607242>
- David A. Wheeler (2017) Flawfinder Home Page. <https://dwheeler.com/flawfinder/>. Accessed 18 Jan 2025
- Devlin J, Chang M-W, Lee K, et al (2019) BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of the 2019 Conference of the North 4171–4186. <https://doi.org/10.18653/V1/N19-1423>
- Feng Z, Guo D, Tang D, et al (2020) CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In: Findings of the Association for Computational Linguistics: EMNLP 2020. Association for Computational Linguistics, Stroudsburg, PA, USA, pp 1536–1547
- Goutte C, Gaussier E (2005) A Probabilistic Interpretation of Precision, Recall and F-Score, with Implication for Evaluation. Lecture Notes in Computer Science 3408:345–359. https://doi.org/10.1007/978-3-540-31865-1_25
- Gürfidan R (2024) VULREM: Fine-Tuned BERT-Based Source-Code Potential Vulnerability Scanning System to Mitigate Attacks in Web Applications. Applied Sciences 2024, Vol 14, Page 9697 14:9697. <https://doi.org/10.3390/APP14219697>
- Hanif H, Maffeis S (2022) VulBERTa: Simplified Source Code Pre-Training for Vulnerability Detection. Proceedings of the International Joint Conference on Neural Networks 2022-July: <https://doi.org/10.1109/IJCNN55064.2022.9892280>

- Haque R, Ali A, Mcclean S, et al (2024) Heterogeneous Cross-Project Defect Prediction Using Encoder Networks and Transfer Learning. *IEEE Access* 12:409–419. <https://doi.org/10.1109/ACCESS.2023.3343329>
- Harzevili NS, Belle AB, Wang J, et al (2023) A Survey on Automated Software Vulnerability Detection Using Machine Learning and Deep Learning. *ACM Comput Surv* 37:. <https://doi.org/10.1145/3699711/ASSET/CF5E99B1-65E5-4D5E-B788-FFDFB5725C05/ASSETS/GRAPHIC/CSUR-2023-0542-U07.JPG>
- IBM (2024) Cost of a data breach 2024 | IBM. <https://www.ibm.com/reports/data-breach>. Accessed 29 Dec 2024
- Kaur A, Nayyar R (2020a) A Comparative Study of Static Code Analysis tools for Vulnerability Detection in C/C++ and JAVA Source Code. *Procedia Comput Sci* 171:2023–2029. <https://doi.org/10.1016/j.procs.2020.04.217>
- Kaur A, Nayyar R (2020b) A Comparative Study of Static Code Analysis tools for Vulnerability Detection in C/C++ and JAVA Source Code. *Procedia Comput Sci* 171:2023–2029. <https://doi.org/10.1016/j.procs.2020.04.217>
- Li X, Xin Y, Zhu H, et al (2023) Cross-domain vulnerability detection using graph embedding and domain adaptation. *Comput Secur* 125:103017. <https://doi.org/10.1016/J.COSE.2022.103017>
- Li Z, Zou D, Xu S, et al (2022) SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities. *IEEE Trans Dependable Secure Comput* 19:2244–2258. <https://doi.org/10.1109/TDSC.2021.3051525>
- Li Z, Zou D, Xu S, et al (2018) VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. 25th Annual Network and Distributed System Security Symposium, NDSS 2018. <https://doi.org/10.14722/ndss.2018.23158>
- Lin G, Wen S, Han QL, et al (2020) Software Vulnerability Detection Using Deep Neural Networks: A Survey. *Proceedings of the IEEE* 108:1825–1848. <https://doi.org/10.1109/JPROC.2020.2993293>
- Lin G, Zhang J, Luo W, et al (2018) Cross-Project Transfer Representation Learning for Vulnerable Function Discovery. *IEEE Trans Industr Inform* 14:3289–3297. <https://doi.org/10.1109/TII.2018.2821768>
- Liu S, Lin G, Qu L, et al (2022) CD-VulD: Cross-Domain Vulnerability Discovery Based on Deep Domain Adaptation. *IEEE Trans Dependable Secure Comput* 19:438–451. <https://doi.org/10.1109/TDSC.2020.2984505>
- Ma W, Zhao M, Soremekun E, et al (2022) GraphCode2Vec: Generic Code Embedding via Lexical and Program Dependence Analyses. *Proceedings - 2022 Mining*

- Software Repositories Conference, MSR 2022 524–536.
<https://doi.org/10.1145/3524842.3528456>
- Marjanov T, Pashchenko I, Massacci F (2022) Machine Learning for Source Code Vulnerability Detection: What Works and What Isn't There Yet. *IEEE Secur Priv* 20:60–76. <https://doi.org/10.1109/MSEC.2022.3176058>
- Nguyen V, Le T, de Vel O, et al (2020) Dual-Component Deep Domain Adaptation: A New Approach for Cross Project Software Vulnerability Detection. In: *Advances in Knowledge Discovery and Data Mining*. Springer, Cham
- Nguyen V, Le T, Tantithamthavorn C, et al (2024) Deep Domain Adaptation With Max-Margin Principle for Cross-Project Imbalanced Software Vulnerability Detection. *ACM Transactions on Software Engineering and Methodology* 33:1–34.
<https://doi.org/10.1145/3664602>
- Zeng P, Lin G, Pan L, et al (2020) Software Vulnerability Analysis and Discovery Using Deep Learning Techniques: A Survey. *IEEE Access* 8:197158–197172.
<https://doi.org/10.1109/ACCESS.2020.3034766>
- Zhang C, Liu B, Xin Y, Yao L (2023) CPVD: Cross Project Vulnerability Detection Based on Graph Attention Network and Domain Adaptation. *IEEE Transactions on Software Engineering* 49:4152–4168. <https://doi.org/10.1109/TSE.2023.3285910>
- Zhou H, Sun G, Fu S, et al (2021) Internet Financial Fraud Detection Based on a Distributed Big Data Approach with Node2vec. *IEEE Access* 9:43378–43386.
<https://doi.org/10.1109/ACCESS.2021.3062467>
- Zhou Y, Liu S, Siow J, et al Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks.
<https://doi.org/10.5555/3454287.3455202>